

Centinela Altur — Elevator pitch (STAR) e índice de tecnologías

HackMTY 2026 · Altur Challenge · Basado en el sistema autocontenido altur_100/

Parte 1 — Elevator pitch con metodología STAR

Duración objetivo: 60–90 segundos. Cuatro bloques (Situación · Tarea · Acción · Resultado) más cierre. Cada bloque equivale a una slide en Canva.

S · Situación (15 s)

Un bot con voz clonada marca al banco.

Guion: ¿Imagine que de pronto una IA clona su voz, que podría hacer con ella en un banco da miedo no?, pues eso pasa todos los días y se hace en minutos, cuando se clona una voz un operador no tiene forma alguna de saber si del otro lado hay alguien o si es una IA.

T · Tarea (15 s)

Decidir humano o IA con voces que nunca vimos.

Guion: Nuestro reto fue lograr que se pudiera saber esto, poder entregar una salida en la cual se pudiera saber rápidamente si en una llamada había alguien detrás o era algo meramente sintético como lo es una IA.

A · Acción (30 s)

Un detector A+B autocontenido (altur_100).

- A · Conversacional (42 features): latencias, solapes, silencios, tiempo al primer turno, ratios de habla, número de turnos — cómo se comporta quien llama.
- B · Acústico (3 señales SSL congeladas): XLS-R 300M + cabeza SLS · WavLM-base-plus + cabeza · Flow-LLR — cómo suena la voz.
- Fusión: HistGradientBoostingClassifier sobre A+B, umbral 0.5.
- Versión rápida: XLS-R a 3 ventanas y sin flow → ~2.7× más veloz.
- Desplegado tres veces: Modal (GPU A10G, pago por uso) · Vultr GPU 24/7 (sin cold start, ~1–2 s) · consola pública en Hugging Face Space.

Guion: Para poder desarrollar esto se nos brindó un dataset con 353 llamadas entre una persona y un agente de IA, sin embargo a veces en vez de un humano era un IA, entonces nosotros vimos algo muy interesante, 1 de estas era el ¿cómo se comporta una llamada? Y también ¿cómo suena la voz de una persona?, con estas dos nos hicimos una pregunta más, ¿podríamos definir estadísticamente la probabilidad de que el que este detrás del celular sea un humano?

R · Resultado (20 s)

Precisión de casi 100 %, en producción.

Guion: Pues la respuesta fue un rotundo si, después de iterar cientos de veces con distintos parámetros llegamos a la conclusión de que con este tipo de llamadas se puede definir de forma exacta la probabilidad de que la persona detrás del celular sea un humano.

Cierre (5 s)

Con Centinela Altur logramos algo increíble, incluso sin conocer la voz de alguien podemos saber si es alguien real o es una clonación hecha con IA

Versión de un solo párrafo (~60 s)

Parte 2 — Índice de tecnologías (altur_100)

Orden alfabético. Para cada tecnología: definición exacta y cómo se usa en el sistema.

Tecnología	Definición	Uso en altur_100
AUC (Area Under the ROC Curve)	Métrica de clasificación binaria igual al área bajo la curva ROC (tasa de verdaderos positivos frente a tasa de falsos positivos para todos los umbrales). Equivale a la probabilidad de que un positivo aleatorio reciba mayor puntuación que un negativo aleatorio; 1.0 es separación perfecta.	Métrica principal de evaluación: AUC 1.0 en train→val y 0.9992 en validación cruzada.
Base64	Esquema de codificación que representa datos binarios como texto ASCII usando un alfabeto de 64 símbolos, 6 bits por carácter.	El audio WAV viaja en el campo audio_base64 del cuerpo JSON de POST /detect.
Docker	Plataforma de contenedores que empaqueta una aplicación con todas sus dependencias en una imagen reproducible que se ejecuta aislada sobre el kernel del host.	vultr/Dockerfile (base pytorch/pytorch:2.3.0-cuda12.1-cudnn8-runtime) y docker-compose.yml con restart: unless-stopped para servir la API en Vultr.
FastAPI	Framework web de Python para construir APIs HTTP, basado en type hints y Pydantic para validación automática y generación de documentación OpenAPI; asíncrono sobre el estándar ASGI.	serve.py expone POST /detect y /health y sirve la consola estática en /console/.
ffmpeg (imageio-ffmpeg)	ffmpeg es una suite de software libre para decodificar, codificar, transcodificar y filtrar audio y video. imageio-ffmpeg es un paquete de Python que distribuye un binario de ffmpeg precompilado y devuelve su ruta.	Fallback para convertir formatos distintos de WAV (mp3, m4a, ogg) a WAV PCM antes de inferir.
Flow-LLR (Normalizing flows + Log-Likelihood Ratio)	Un normalizing flow es un modelo generativo que transforma una distribución simple (gaussiana) en una compleja mediante una secuencia de funciones invertibles y diferenciables, lo que permite calcular la log-verosimilitud exacta de una muestra. La razón de log-verosimilitud (LLR) compara la verosimilitud bajo dos modelos (aquí: humano frente a sintético).	flow_llr.joblib: dos flows NSF (zuko) entrenados sobre embeddings WavLM (StandardScaler + PCA), uno por clase; el LLR pasa por una regresión logística para dar probabilidad. Desactivado en la versión rápida.
HistGradientBoostingClassifier (scikit-learn)	Clasificador de gradient boosting sobre árboles de decisión que discretiza las features en histogramas de hasta 255 bins para acelerar el entrenamiento, inspirado en LightGBM; soporta valores faltantes de forma nativa.	Modelo de fusión final altur_ab.joblib: max_iter=400, learning_rate=0.05, max_depth=3, l2_regularization=1.0, entrenado sobre las 42 features conversacionales + 3 señales de audio.
Hugging Face Hub / Transformers	Hugging Face Hub es un repositorio público de modelos y datasets. Transformers es la biblioteca de Python que implementa arquitecturas Transformer preentrenadas y las descarga del Hub con from_pretrained.	Carga de facebook/wav2vec2-xls-r-300m y microsoft/wavlm-base-plus con AutoFeatureExtractor, Wav2Vec2Model y WavLMModel; caché en HF_HOME o en un Volume.
Hugging Face Space (Static)	Servicio de hosting de aplicaciones de Hugging Face. El SDK Static sirve archivos HTML/JS/CSS sin backend.	hf_space/index.html: consola pública que envía el audio al backend GPU

		definido en <code>window.API_BASE</code> .
Holdout	Partición de los datos que se separa antes de entrenar y nunca se usa para ajustar el modelo, con el fin de estimar el rendimiento en datos no vistos.	<code>train_holdout.py</code> reserva 71 llamadas (<code>holdout_test.csv</code>) → 98.6 % de acierto.
joblib	Biblioteca de Python para serializar objetos (especialmente arrays NumPy y modelos de scikit-learn) de forma eficiente y para paralelizar tareas.	Persistencia de <code>altur_ab.joblib</code> , <code>wavlm_head.joblib</code> , <code>flow_llr.joblib</code> y <code>prosody_clf.joblib</code> .
Modal	Plataforma serverless para ejecutar código Python en la nube con GPU bajo demanda; define imágenes, funciones, volúmenes persistentes y endpoints web desde Python, escalando a cero cuando no hay uso.	<code>modal_serve.py</code> : función con <code>gpu="A10G"</code> , Volume <code>altur-hf-cache</code> para los pesos, <code>scaledown_window</code> de 1200 s y endpoint web <code>/detect</code> .
MLP (Multi-Layer Perceptron)	Red neuronal feed-forward de capas totalmente conectadas con activaciones no lineales (ReLU) entre ellas.	Cabeza de XLSRSLs: LayerNorm → Linear → ReLU → Dropout → Linear → ReLU → Dropout → salida.
NumPy / pandas	NumPy: biblioteca de arrays n-dimensionales y operaciones numéricas vectorizadas. pandas: estructuras DataFrame/Series para datos tabulares.	Manejo de señales de audio (NumPy) y de la tabla de features/manifest (<code>pandas</code>), incluido <code>feats_cache.csv</code> .
NVIDIA A10G	GPU de centro de datos de NVIDIA (arquitectura Ampere, 24 GB GDDR6) orientada a inferencia y gráficos.	GPU seleccionada en Modal para correr XLS-R y WavLM.
Praat / parselmouth	Praat es un programa de análisis fonético. <code>parselmouth</code> es su binding de Python, que expone sus algoritmos (pitch, jitter, shimmer, armonicidad).	Cálculo de biomarcadores de voz (jitter local, shimmer local, HNR, F0, fracción sonora) para <code>prosody_clf.joblib</code> .
Pydantic	Biblioteca de validación de datos de Python basada en type hints que valida y serializa modelos de datos.	Modelo <code>DetectRequest{audio_base64}</code> de FastAPI.
PyTorch / torchaudio / torchvision	PyTorch: framework de deep learning con tensores y diferenciación automática. torchaudio: E/S y transformaciones de audio (p. ej. Resample). torchvision: modelos y utilidades de visión.	Ejecución de XLS-R, WavLM y las cabezas; resampleo 8 kHz → 16 kHz; <code>torchvision.models</code> para la ResNet de respaldo.
ResNet (Residual Network)	Arquitectura de red convolucional con conexiones residuales (skip connections) que suman la entrada de un bloque a su salida, permitiendo entrenar redes muy profundas.	<code>VoiceSpoofResNet</code> (<code>audio_model.py</code>): CNN sobre espectrograma mel, usada solo como respaldo si WavLM no está disponible.
scikit-learn	Biblioteca de aprendizaje automático de Python con algoritmos de clasificación, regresión, clustering, preprocesamiento, selección de modelos y métricas.	<code>HistGradientBoostingClassifier</code> , <code>LogisticRegression</code> , <code>StandardScaler</code> , <code>PCA</code> , <code>StratifiedKFold</code> , <code>cross_val_predict</code> , <code>roc_auc_score</code> , <code>accuracy_score</code> .
Self-Supervised Learning (SSL) de voz	Paradigma en el que un modelo aprende representaciones a partir de audio sin etiquetas, prediciendo partes enmascaradas o cuantizadas de la señal (p. ej. <code>wav2vec 2.0</code> , WavLM). Las capas resultantes codifican información fonética, prosódica y de canal.	XLS-R y WavLM se usan congelados (sin actualizar sus pesos) como extractores de features; solo se entrenan cabezas pequeñas encima.
Sensitive Layer Selection (SLS)	Técnica de anti-spoofing que toma las salidas de todas las capas de un modelo SSL y aprende un peso por capa para combinar las que más discriminan voz real de sintética.	XLSRSLs: <code>layer_w</code> (<code>n_layers=25</code>) pondera las medias temporales de los 25 estados ocultos de XLS-R antes del MLP.
soundfile	Biblioteca de Python (sobre <code>libsndfile</code>) para leer y escribir archivos de audio como arrays NumPy.	Lectura del WAV recibido (<code>always_2d</code> , <code>float32</code>) y de los audios del dataset.
Validación cruzada estratificada (StratifiedKFold)	Método de evaluación que divide los datos en k pliegues preservando la proporción de clases; se entrena k veces dejando un pliegue fuera cada vez y se promedian las métricas.	5-fold × 5 semillas → 99.26 % ± 0.23.
uvicorn	Servidor ASGI de alto rendimiento para Python (basado en <code>uvloop</code> y <code>httptools</code>) que ejecuta aplicaciones asíncronas como FastAPI.	Comando de arranque: <code>uvicorn serve:app --host 0.0.0.0 --port 80</code> .
VAD (Voice Activity Detection)	Algoritmo que decide, por tramas cortas, si hay voz o silencio en una señal.	<code>turns_from_audio</code> deriva los turnos de cada canal: <code>webrtcvad</code> (Google WebRTC, agresividad 3, tramas de 30 ms) con fallback a VAD por energía; en el despliegue se usa a propósito el de energía, el mismo con el que se entrenó.
Vultr Cloud GPU	Proveedor de infraestructura en la nube que ofrece instancias con GPU NVIDIA (A16/A40/L40S) facturadas por	Alternativa a Modal: contenedor 24/7 sin cold start (~1–2 s por request),

	tiempo.	instalado con vultr/setup.sh.
WAV / PCM	WAV es un contenedor de audio de Microsoft/IBM; PCM (Pulse-Code Modulation) es la representación cruda de muestras. En el reto: estéreo, 8 kHz, 16 bits.	Entrada del sistema: canal 0 = quien llama (se clasifica), canal 1 = agente; mono también se acepta.
WavLM (WavLM-base-plus)	Modelo SSL de voz de Microsoft que extiende HuBERT con predicción de unidades enmascaradas y denoising, aprendiendo representaciones útiles para tareas más allá del reconocimiento (hablante, diarización).	microsoft/wavlm-base-plus congelado → mean/std del último estado oculto → cabeza (scaler + clasificador) en wavlm_head.joblib. Es la señal wavlm.
XLS-R (wav2vec2-xls-r-300m)	Modelo SSL multilingüe de Meta basado en wav2vec 2.0, con 300 M de parámetros, preentrenado en 436 000 h de audio en 128 idiomas.	facebook/wav2vec2-xls-r-300m congelado con output_hidden_states → 25 capas → XLSRSLs (xlsr_sls.pt). Es la señal xlsr; en la versión rápida se promedian 3 ventanas de 6 s.
zuko	Biblioteca de PyTorch para normalizing flows (MAF, NSF, etc.) con API modular.	zuko.flows.NSF (Neural Spline Flow) como modelo de densidad de cada clase en FlowLLR.

Resumen de features conversacionales (señal A)

42 features derivadas de los turnos por canal: latencias de respuesta del que llama (media, desviación, mínimo, máximo), solapamientos, silencios, tiempo hasta el primer turno del que llama (time_to_first_caller_s), ratios de habla caller/agente, número y duración de turnos, y duración total de la llamada.